

DeepSearcher

第二阶段学习讲义

手机阅读版 · 全栈链路、评测与系统设计

学习范围	FastAPI、工作台、Trace、评测、并发、安全、系统设计
学习方式	第一遍直接理解，第二遍模拟面试
生成日期	2026-08-18

适合手机竖屏阅读；目录与 PDF 书签可直接跳转章节。

目录

1. FastAPI 业务入口：把 HTTP 请求接到 DeepSearcher	3
2. 用户工作台与文件上传：产品链路和兼容链路	16
3. Trace 全栈链路：应用事件不是模型思维链	18
4. React 状态与测试：产品工作台与学习控制台并存	19
5. RAG 评测：把“看起来不错”变成可重复实验	20
6. 延迟、Token 与成本：分解而不是只看总数	21
7. Dense、BM25、Hybrid 与 RRF	22
8. 并发、安全与十倍流量	23
9. 生产系统设计与 SSE	25
10. 项目表达：结论、因果、证据、取舍、边界	26
11. 第二阶段不能说错的事实	28

范围：AI 全栈后端、可观测性、前端数据流、RAG 评测、并发、安全与系统设计。当前进度：第二阶段已完成并封存；先完成第三阶段可信知识治理与企业入库学习，再进入第二遍模拟面试。用途：承接第一阶段 RAG 与 Agent 基础，继续第一遍详细学习，避免单个阶段文档过长。

前置材料：第一阶段学习讲义·第二阶段面试问答复习手册·项目问题与优化清单 状态更新（2026-08-18）：本册中的 **BackgroundTasks**、**PDF-only**、**无产品 RBAC**、**无业务金标**和 2026-08-02 测试数字均为历史基线。当前入库由独立 Worker 和 **IngestJob** 租约恢复驱动，产品已具备 **Workspace/KB ACL**；进阶更新统一见第三阶段学习讲义。

1. FastAPI 业务入口：把 HTTP 请求接到 DeepSearcher

1.1 先记住一条中文链路

客户端 HTTP 请求

- FastAPI 解析并校验参数
- `main.py` 选择配置、入库或查询函数
- `RuntimeRegistry` 按租户发放请求作用域 Searcher 租约
- `RAGRouter` 选择 `DeepSearch`、`ChainOfRAG` 或 `NaiveRAG`
- `LLM`、`Embedding`、`Milvus` 完成查询
- `main.py` 把结果或异常转换成 HTTP 响应

FastAPI 解决的是“怎样把 Python 业务能力安全、稳定地暴露成网络接口”，它本身不负责 RAG。

`main.py` 是 HTTP 适配层，`deepsearcher/online_query.py` 是很薄的业务门面，Agent、Embedding 和 Milvus 才是查询主体。

1.2 应用启动时发生什么

`main.py` 导入时只定义应用工厂与路由，不连接模型或向量库。FastAPI lifespan 启动后读取配置，创建 `RuntimeRegistry`，再构建当前活动版本需要的：

- LLM 客户端。
- Embedding 客户端。
- 文件加载器和网页爬虫。
- VectorDB 客户端。
- 由 DeepSearch 和 ChainOfRAG 组成的默认 RAGRouter。
- 作为基础能力保留的 NaiveRAG。

在线请求通过 `Depends(get_runtime)` 获得租约，退出时释放；版本切换后旧实例会等最后一个旧请求结束再关闭。初始化失败不会让应用进程消失，`/health` 与业务接口会返回带安全组件名的 503，关闭阶段统一等待 SSE 清理任务并释放向量客户端。

旧版本还有两个容易忽略的事实；当前 FastAPI 已完成版本化改造：

1. `init_config()` 的模块级对象仍为 CLI/库调用保留，但 FastAPI 在线请求从 `lifespan` 中的 `RuntimeRegistry` 取请求租约。
2. 热更新不写回 `config.yaml`；它作为版本 patch 写入共享控制库，同一部署的 Uvicorn worker 会在下一请求读取活动版本，并支持回滚。

1.3 四个业务接口分别做什么

接口：POST `/set-provider-config/`

输入位置：JSON Body

当前作用：为当前受信租户构建并发布下一 runtime 版本

关键边界：旧请求继续使用旧租约；密钥不回显且只能引用环境变量

接口：POST `/load-files/`

输入位置：JSON Body

当前作用：读取后端机器上的文件路径，切 Chunk、Embedding 后写入 Milvus

关键边界：不是浏览器二进制上传接口；默认 `batch_size=256`

接口：POST `/load-website/`

输入位置：JSON Body

当前作用：抓取 URL、切 Chunk、Embedding 后写入 Milvus

关键边界：依赖爬虫和外部网络；当前入口没有暴露抓取超时等治理参数

接口: `POST /query`

输入位置: JSON Body

当前作用: 调用当前租户 RAGRouter 查询, 可选返回 Trace

关键边界: 问题不进入 URL; 最长 4000 字, `max_iter` 为 1-10

接口: `POST /query/stream`

输入位置: JSON Body + SSE

当前作用: 增量返回受控执行阶段与最终结果

关键边界: 阶段不是思维链; 断连协作取消并释放 runtime 租约

Pydantic 模型 `ProviderConfigRequest` 会校验 `feature`、`provider` 和 `config` 三个字段是否存在以及基本类型。其他接口通过 `Body()` 声明参数。比如 `max_iter` 设置了 `ge=1, le=10`, 传入 0 时请求会在进入业务函数之前被 FastAPI/Pydantic 拒绝, 并返回安全的 422 错误信封。

但“类型校验通过”不等于“业务输入安全”。例如:

- `feature` 只是普通 `str`, 是否受支持要到 `set_provider_config()` 才判断。
- `paths` 仍是服务端路径, 因此入口必须使用服务令牌, 不能直接暴露给浏览器或不受信调用方。
- Collection 名、产品 Collection 范围和精确确认头在 API 层校验, 租户上下文还会再次检查白名单。
- `original_query` 最长 4000 字, `max_iter` 最多 10; 这只是请求预算, 不等于完整成本治理。
- 一键启动会给默认 `local` 租户注入服务令牌; 非默认租户还必须通过同一令牌与服务端

Collection 白名单。

CORS 现在由应用工厂统一加载，默认关闭。只有 `DEEPSEARCHER_CORS_ORIGINS` 或 CLI 重复 `--cors-origin` 明确列出的 HTTP(S) origin 会被允许；*、带用户信息、路径、查询或 fragment 的值会被忽略，method/header 也采用白名单。因此 `uvicorn main:app` 与直接运行脚本不会再得到两套相反策略。

1.4 一个查询请求怎样进入 DeepSearcher

假设客户端请求：

```
POST /query
Content-Type: application/json
X-DeepSearcher-Service-Token: <service-token>

{"original_query": "Milvus 适合什么场景",
 "max_iter": 3, "include_trace": true}
```

完整执行顺序是：

1. FastAPI 从 JSON Body 解析字段，并检查问题长度与 $1 \leq \text{max_iter} \leq 10$ 。
2. `perform_query()` 看到 `include_trace=True`，调用 `query_with_trace(original_query, max_iter)`。
3. 请求依赖从 `RuntimeRegistry` 获得当前租户的不可变 runtime 租约，`query_with_trace()` 使用租约里的 `RAGRouter` 创建安全 `TraceCollector`。

4. RAGRouter 先让 LLM 在 DeepSearch、ChainOfRAG 与 NaiveRAG 中选择一个 Agent; 未显式配置时 NaiveRAG 是 fallback/default agent。
5. 被选中的 Agent 做 Collection 路由、Embedding、Milvus 检索和答案生成, 并把受控应用事件写入 Collector。
6. `query_with_trace()` 返回 (`answer`, `retrieval_results`, `consume_tokens`, `trace`)。
7. `perform_query()` 丢弃第二项检索结果, 只向客户端返回 `result`、`consume_token` 和 `trace`。

未要求 Trace 时, 入口调用 `query()`, 响应保持原有结构:

```
{
  "result": ".....",
  "consume_token": 1234
}
```

`online_query.py` 没有在导入时复制 `default_searcher`, 而是在每次调用时读取 `configuration.default_searcher`。这使配置切换后新请求能拿到新对象, 但也带来并发代际问题: 请求 A 可能已经拿到旧 Searcher, 请求 B 重建全局对象后再拿到新 Searcher。单次调用中的局部变量仍指向旧对象, 不会凭空变成新对象, 但系统会暂时存在新旧两套客户端, 且切换过程没有锁、版本号 and 原子发布协议。

1.5 文件入库请求怎样执行

`POST /load-files/` 接收的是一个后端可见路径或路径列表，而不是文件字节。入口把参数交给 `load_from_local_files()` 后，执行：

路径存在性检查

- 文件或目录解析为 Document
- 按 1500/100 默认参数切 Chunk
- 按 `batch_size` 批量生成 Embedding
- 写入指定或默认 Collection

例如请求体：

```
{
  "paths": ["D:/knowledge/a. pdf", "
D:/knowledge/b. pdf"],
  "collection_name": "interview_notes",
  "batch_size": 64
}
```

这里的路径必须存在于 FastAPI 服务所在机器。浏览器用户电脑上的 `C:/Users/.../a. pdf` 对部署在另一台服务器上的后端没有意义。这正是后续本地代理需要先接收浏览器文件、保存临时文件，再把“后端可见路径”传给该接口的原因。

当前入口没有暴露 `chunk_size`、`chunk_overlap` 和 `force_new_collection`，因此 API 入库固定使用底层函数默认的 `1500/100`，并默认不删除 Collection。这减少了误操作入口，但也降低了调参灵活性。

1.6 `def` 与 `async def` 在这里有什么区别

当前四个路由函数都是普通 `def`。在本项目锁定的 FastAPI 0.115.12 / Starlette 0.46.2 中，FastAPI 会在线程池中执行同步路由，避免它们直接占住主事件循环。

必须区分三句话：

1. `def` 会被放入线程池：其他异步连接仍有机会被事件循环处理。
2. 线程池不等于无限并发：一个长时间等待 LLM、Embedding 或 Milvus 的请求会持续占用一个工作线程；线程耗尽后新请求仍要排队。
3. 把签名改成 `async def` 不会自动变快：如果函数内部仍直接调用同步 `llm.chat()`、`embed_query()` 和 Milvus 客户端，反而会直接阻塞事件循环。

正确选择取决于依赖：

- 依赖只有同步客户端：保留 `def`，或显式放入容量受控的线程池，并配置超时与并发上限。
- 依赖提供真正异步客户端：使用 `async def`，每层都 `await`，并把取消和 deadline 向下传播。
- PDF 解析等 CPU 密集任务：线程受 Python GIL 和 CPU 竞争影响，更适合任务队列或进程池，而不是占用 Web 请求线程。
- 大批量入库：更适合提交后台任务并返回 task ID，由 Worker 完成解析、Embedding 和写库。

当前 DeepSearch 内部虽然出现 `asyncio.gather()`，但其中仍包含同步模型和向量库调用，所以不能把整个系统描述成“异步高并发架构”。

1.7 异常怎样变成 HTTP 响应

当前错误协议分三层：Pydantic 校验错误统一为 422/`INVALID_REQUEST`；运行时、向量库和 API 领域异常映射为稳定业务码；未分类异常统一为 500/`INTERNAL_ERROR` 或查询专用 `QUERY_FAILED`。响应和日志都不使用原始 `str(e)`，Provider 更新也不回显 config。

当前主要映射包括：

场景	建议状态	稳定错误码示例
参数语义不合法	400/422	<code>INVALID_ARGUMENT</code>
文件不存在	404	<code>SOURCE_NOT_FOUND</code>
Collection 状态冲突	409	<code>COLLECTION_CONFLICT</code>
上游模型限流	429 或受控 503	<code>UPSTREAM_RATE_LIMITED</code>
LLM、Embedding、Milvus 暂时不可用	502/503	<code>DEPENDENCY_UNAVAILABLE</code>
请求超过 deadline	504	<code>QUERY_TIMEOUT</code>
未分类内部错误	500	<code>INTERNAL_ERROR</code>

客户端响应只返回稳定 `code`、安全 `message`、`request_id` 和 `retryable`，响应头回传同一 `X-Request-ID`。未知异常日志只记录请求 ID、方法、路由模板和异常类型，不记录异常文本；如需完整堆栈，必须先建立结构化字段脱敏与受限日志访问，不能直接把可能含密钥的 `traceback` 打进普通日志。

1.8 相关 API 测试实际保护了什么

`tests/test_query_api.py` 现在通过真实 `TestClient` 生命周期保护 POST 查询、SSE、租户/Collection 隔离、Provider 版本发布与回滚、请求 ID、限流、CORS 和安全错误映射。验收覆盖 400/401/404/409/422/429/503/504/500，并扫描响应和错误日志中是否出现测试密钥与本机路径。

这些 `TestClient` 用例会经过 ASGI 路由、`Pydantic`、`middleware`、`exception handler` 和 `lifespan`，并用 `Stub runtime` 隔离外部依赖。它们不连接真实供应商，因此真实 Milvus/模型另由集成脚本验证；容量、长时间资源泄漏、多个真实 worker 的限流一致性仍需要压测和故障注入。

合理的测试分层是：

1. 纯函数单元测试：直接测试错误分类和响应 DTO。

2. FastAPI `TestClient`/ASGI 测试：覆盖 200、422、404、503、504 和响应脱敏。
3. 集成测试：使用可控的测试 Milvus 与模型 Stub，验证完整调用链。
4. 并发测试：配置切换与查询同时发生时，验证版本隔离和资源回收。

1.9 为什么当前方案仍有价值，以及替代方案的代价

当前单文件入口适合学习和本地 Demo：代码少、调用链直、Swagger 文档可自动生成，也容易从函数跳进业务代码。若过早拆成很多 Controller、Service、Repository，反而会增加理解成本。

但当目标变为多人、多租户或生产服务时，需要付出结构化成本：

- lifespan 和依赖注入增加代码层次，但换来可测试的资源生命周期。
- 任务队列增加部署组件和状态管理，但避免大文件入库拖垮 Web 线程。
- 领域错误模型需要维护映射表，但客户端才能可靠重试和提示。
- 配置中心或版本化配置增加基础设施，但多 worker 和多实例才能一致。
- 认证、路径白名单与密钥脱敏增加开发工作，却是公开服务的基本安全边界。

1.10 常见错误说法

错误说法	正确说法
FastAPI 接口是 <code>def</code> ，所以一定阻塞整个服务	同步路由通常在线程池执行，不直接阻塞事件循环，但会占用有限工作线程
改成 <code>async def</code> 就能提高并发	只有整条依赖链真正异步并正确 <code>await</code> 才可能改善并发
<code>/load-files/</code> 是文件上传接口	它接收后端文件系统路径，不接收文件字节
配置接口修改了 YAML	它只修改当前进程内存并重建全局对象
所有错误返回 500 很统一	这会混淆客户端错误、依赖故障和服务 Bug，也不利于重试
有两个 monkeypatch 测试就证明 API 可用	当前测试只覆盖函数返回形状，没有覆盖真实 HTTP 和依赖生命周期
使用多个 Uvicorn worker 就能共享新配置	每个 worker 是独立进程，内存配置不会自动同步

1.11 面试标准表达

DeepSearcher 的 FastAPI 入口是 HTTP 适配层。导入 `main.py` 不连接外部依赖，`lifespan` 创建 `RuntimeRegistry`，每个请求通过依赖注入获得当前租户的不可变 `runtime` 租约。同步 `POST /query` 与 SSE `POST /query/stream` 都使用请求作用域 `Searcher`，由 `RAGRouter` 在 `DeepSearch`、`ChainOfRAG` 和 `NaiveRAG` 之间选择，再返回答案、Token、可选安全 Trace 或增量阶段。同步模型和向量 SDK 仍占用线程资源，因此每进程限流只是保护层；产品层已有 `Workspace/KB RBAC` 和可恢复入库 Worker，生产化仍需要外部 `deadline`、分布式配额、外部身份和通用任务编排。

1.12 当前项目的诚实边界

- 已有可读的配置、文件入库、网页入库、查询和可选 Trace 接口。
- 已有 FastAPI/Pydantic 参数校验、服务令牌、租户与 Collection 授权，以及产品层 `Workspace/KB RBAC`；尚无外部身份提供商和完整 Organization 治理。
- 当前是同步阻塞依赖加线程池承载，不是端到端异步架构。
- 配置使用共享 SQLite 控制面、CAS 发布和请求租约延迟回收，多 worker 会在请求边界观察活动版本。

- 核心文件接口仍读取服务端路径，但必须使用服务令牌；浏览器上传由 BFF 校验并持久化后再调用。
- 业务错误已有稳定安全信封与请求 ID；未知日志不记录异常文本，深入排障仍需受控脱敏日志系统。
- API 测试覆盖生命周期、鉴权、租户隔离、POST/SSE、错误状态、限流、CORS 和泄漏扫描。
- CORS 默认关闭，只允许配置中的精确可信 origin；仍需由生产网关配合 TLS 与边界策略。
- 没有负载测试数据，不能声称当前接口已经达到生产级吞吐、可用性或安全性。

2. 用户工作台与文件上传：产品链路和兼容链路

2.1 当前产品工作台链路

浏览器选择受支持文件

- product-api.ts 用 FormData 发送 multipart
- /api/knowledge-bases/{id}/documents 读取并校验
- 校验扩展名与 MIME/容器真实性，计算 SHA-256 并拒绝重复上传
- 保存 Document、IngestJob 和持久文件，状态为 queued
- 返回 202，独立 Worker 认领 IngestJob 并置为 processing
- 用服务端文件路径调用 DeepSearcher /load-files/
- 成功变为 ready，失败变为 failed
- React 在 queued/processing 时每两秒轮询

`product-api.ts` 不手工设置 `multipart` 的 `Content-Type`，而是让浏览器根据 `FormData` 生成带 `boundary` 的 `Header`。服务端使用文件名、`Content-Type`、20 MiB、`%PDF` 魔数和 `SHA-256` 做基础校验；`Path(display_name).name` 去掉目录部分，实际存储名使用内部 `Document ID`。

当前上传支持 `PDF`、`Excel`、`PPTX`、图片及注册表允许的格式，`MIME`/容器检查防止扩展名伪装；入库任务由独立 `Worker` 以租约、重试和恢复机制处理，线性 `Parse/Chunk/Embed/Index` pipeline 记录节点级失败。它仍不是通用 `DAG` 或大文件直传平台：超大文件、病毒扫描、对象存储直传和跨区域队列仍需独立设计。

2.2 `/console` 旧学习控制台兼容链路

浏览器选择 `PDF`

- `api.js` 转成 `Base64 JSON`
- `/api/ingest` 校验并写入 `TemporaryDirectory`
- 把临时服务端路径交给 `/load-files/`
- `DeepSearcher` 在请求返回前读完
- 请求结束后自动清理临时目录

这条链路仍有学习价值：它解释了浏览器文件与服务端路径的边界，也展示了为什么 `BFF` 能适配旧接口。但它不再是默认产品页面的上传方式。面试时必须明确“产品页是 `multipart` + 持久状态，`/console` 才是 `Base64` + 临时目录”。

3. Trace 全栈链路：应用事件不是模型思维链

3.1 数据怎样流动

```
query_with_trace()
→ 创建请求级 TraceCollector
→ RAGRouter 记录 Agent
→ ChainOfRAG 逐轮记录子问题、Collection、检索文档、
  中间答案、支持文档、反思和 Token
→ build() 生成版本化安全 JSON
→ 学习控制台: FastAPI → /api/query → api.js →
  /console 的 TracePanel
→ 产品工作台: Trace → Citation 持久化 →
  product-api.ts → 引用抽屉
```

Trace 记录程序显式可见的输入、输出和阶段事件，不读取也不声称暴露模型隐藏思维链。当前脱敏策略包括：最多展示 5 条文档、每条 600 字，不返回 Embedding 和原始 metadata；本地路径只保留文件名，URL 删除查询参数，引用 metadata 使用白名单。

NaiveRAG、ChainOfRAG 和 DeepSearch 都已接入安全阶段事件。当前完整执行 Trace 的版本为 v7；产品问答 SSE 使用独立的 v1 公开事件契约，增量展示允许公开的路由、检索、支持和反思统计；旧 `/console` 的 `/api/query` 仍在查询完成后用 `build()` 返回完整 JSON Trace。两条链路都只展示应用事件，不是模型隐藏思维链。

4. React 状态与测试：产品工作台与学习控制台并存

当前主入口已经是 TypeScript 产品工作台 `App.tsx`，由 React Router 提供四组产品路由：

<code>/</code>	新对话
<code>/chat/:conversationId</code>	持久化对话与引用抽屉
<code>/knowledge</code>	知识库列表
<code>/knowledge/:id</code>	文档上传、状态轮询和失败重试
<code>/console</code>	原学习控制台

产品工作台使用 TanStack Query： `useQuery` 管理知识库、文档和对话缓存， `useMutation` 管理创建、上传与重试，成功后通过 `invalidateQueries` 让服务端状态重新成为事实来源。文档处于 `queued/processing` 时每两秒轮询，至少有一份 `ready` 文档后才允许提问。问答通过 `FetchReadableStream` 消费 SSE，阶段增量展示； `AbortController` 可以停止本次回答，完成后再刷新持久化 `Conversation`。

旧学习控制台保留在 `/console`。它仍通过 `api.js` 调用 `/api/query`，一次写入答案、延迟、Token 和 Trace，并用 `TracePanel` 回放应用事件。因此面试时必须区分“产品主页面”和“调试学习控制台”，不能再把两者统称为一个页面。

现有组件测试保护默认工作台、创建对话、恢复回答、重新生成、本地反馈、引用抽屉的展开/可用

状态、移动端焦点闭环、文档表格语义和 /console 回退入口；API 与 Trace 测试保护旧控制台的字段映射和展示。真实浏览器又定向验证了响应式回流、焦点、可见正文对比度抽样和紧凑操作目标。它们仍不能证明真实 LLM、Milvus、所有浏览器、并发竞态、屏幕阅读器矩阵或完整 WCAG 合规。

5. RAG 评测：把“看起来不错”变成可重复实验

完整评测需要分三层：

检索层：金标证据是否进入 TopK
生成层：给定证据后答案是否正确、忠实、可引用
系统层：延迟、Token、成本、失败率和稳定性

仓库自带 `evaluation/evaluate.py`，使用 2WikiMultiHopQA 比较 DeepSearcher 与 Naive RAG 的 `Recall@2/Recall@5`，并统计 DeepSearcher Token 和流水线错误。它从 `supporting_facts` 中提取文档标题，因此主要是标题级检索评测；脚本没有使用标准答案字段，也就没有评最终答案正确性、Faithfulness、引用正确性和拒答能力。

第一版业务黄金集可以先做 20 题，覆盖单跳、多跳、比较、精确实体、不可回答和冲突问题。每题至少保存：

```
question
reference_answer
answerable
expected_sources / chunk_ids
question_type
corpus_version
```

常用检索指标包括 Recall@K、MRR、nDCG、重复率和空结果率；生成指标包括 Answer Correctness、Faithfulness、Citation Correctness 和拒答准确率。诊断时先判断金标证据是否进入 TopK：未进入是检索问题；进入后仍答错，主要检查生成和上下文组织。

6. 延迟、Token 与成本：分解而不是只看总数

代理通过 `perf_counter()` 测量“代理准备调用上游到完整响应解析完成”的近似端到端延迟。它包含 FastAPI、LLM、Embedding 和 Milvus 等等待，却不包含完整浏览器网络和 React 渲染，也没有各阶段 Span。

平均延迟会被大量快请求稀释，生产系统至少报告 p50、p95、p99、超时率和成功率。180 秒代理超时只是失败边界，不是 SLO。

当前 Token 来自每次 `ChatResponse.total_tokens` 的手工累加：包括 Agent 路由、Collection 路由、子问题、中间答案、支持文档筛选、可选反思和最

终回答。它没有拆分 input/output/cached/reasoning Token，也没有统计 Embedding 使用量，所以只能近似估算费用。

ChainOfRAG 在单 Collection、`max_iter=3` 的常见路径中大约需要一次 Agent 路由、每轮三次 LLM 调用和一次最终回答，即约 11 次调用。后续轮次还会携带累积上下文，因此 Token 和延迟不保证严格线性增长。优化目标应是质量、p95、Token、失败率的 Pareto 折中，而不是只追求最高正确率。

7. Dense、BM25、Hybrid 与 RRF

Dense: 根据 Embedding 找语义近似

BM25: 根据词频、稀有度和文档长度找字面匹配

Hybrid: 同时执行两路召回

RRF: 按名次而不是原始分数融合

Dense 擅长同义词和自然语言改写，但对错误码、SKU、函数名和版本号可能不稳定；BM25 擅长稀有精确字符串，却不能普遍理解同义表达。RRF 使用 $\sum 1/(k + \text{rank})$ 融合排名，避免直接比较 L2 与 BM25 这两种方向、范围和量纲都不同的分数。

Milvus 实现已经具备 Analyzer、`sparse_vector`、BM25 Function、稀疏倒排索引、双路 `AnnSearchRequest` 和 `RRFRanker()`。但当前 YAML 没有启用 `hybrid`，实际仍是 Dense-only。已有

Collection 也不能只靠开关升级：它缺少稀疏字段、Function 和索引，需要新建 Schema 并重新入库。

当前 `RetrievalResult.score` 承接 Milvus 的 `distance`，默认 L2 越小越近，而前端笼统标为“相似度”不够严谨。生产响应应同时携带 `metric/score_type/higher_is_better/retrieval_mode`。

8. 并发、安全与十倍流量

并发流量会被 Agent 放大。10 个用户请求在 ChainOfRAG 下可能形成约百次 LLM 调用需求，真正瓶颈可能是线程池、RPM/TPM、Milvus、内存和排队，而不只是 FastAPI。

同步 `def` 路由通常由 FastAPI 放入线程池，不直接阻塞事件循环，但每个长查询会长期占用有限线程。把签名改成 `async def` 不能把同步 LLM、Embedding 和 Milvus SDK 自动变成非阻塞。

模块级共享客户端不一定错误：只读、无请求状态且支持并发的客户端可以复用连接。旧风险在于逐个替换 globals 且没有原子版本；当前 API 使用共享版本控制面和 worker 本地不可变 runtime 缓存，旧版本按请求租约延迟回收。

当前高优先级安全边界包括：

1. 默认本地接口仍未形成最终用户登录与角色体系；当前完成的是服务到服务租户令牌、Collection 授权和每进程查询限流。
2. `/load-files/` 接受服务器路径，因此强制服务令牌且只能由 BFF/受信运维调用，不能直接公开给浏览器。
3. `collection_names` 已受服务端租户白名单约束，但产品用户级身份体系仍需继续建设。
4. 核心与代理都限制 `max_iter ≤ 10`，但仍需总 deadline、模型 Token 预算和分布式并发控制。
5. 租户发布/回滚要求管理令牌，默认租户 Provider 更新要求服务令牌，配置不回显密钥；后续仍需角色和审计治理。
6. URL 抓取可能产生费用滥用、SSRF 和恶意内容入库风险。
7. 查询已经改用 POST Body；仍要避免在自定义日志、APM payload 或错误上报中记录请求体。

十倍流量的改造顺序应是：入口鉴权限流和请求体上限 → 有界并发与背压 → 入库任务队列 → 外部依赖超时、预算和熔断 → 租户授权 → 最后再水平扩容。单纯增加 Uvicorn worker 不能提高模型限额，还会复制连接和进程内配置。

9. 生产系统设计与 SSE

当前 `query_with_trace()`、代理 `response.json()` 和浏览器 `response.json()` 都要等待完整响应。升级路线应先定义事件协议，再改传输：

POST /query-runs

- 鉴权、参数校验、Idempotency-Key
- 创建 run/message/outbox
- Query Worker 执行 Agent
- 事件先写 Event Store，再发布
- GET /query-runs/{id}/events 使用 SSE
- DELETE /query-runs/{id} 显式请求取消

建议事件包括 `run.started`、`routing.completed`、`iteration.started`、`subquery.generated`、`retrieval.completed`、`support_filter.complete`d、可选 `reflection.completed`、`final_answer.completed`，以及唯一终态 `run.completed/run.failed/run.cancelled`。

每个事件携带 `schema_version/run_id/trace_id/event_id/timestamp/payload`。SSE 重连可能重放事件，前端必须按 `event_id` 幂等归并；答案 Delta 需要 `chunk_index` 或 `offset`，不能盲目字符串追加。

客户端断线不等于任务取消。持久化会话通常让任务继续，刷新后按 `Last-Event-ID` 恢复；用户点击停止才发送取消请求。同步 LLM 调用不一定能立

即中断，所以要区分 `cancellation_requested` 和 `cancelled`。

当前主页面已接入 KnowledgeBase、Document、IngestJob、Conversation、Message 和 Citation 的持久状态模型，并在 Workspace/KB ACL 下访问。产品上传使用 multipart、受控 Loader 和独立 Worker；旧 `/console` 上传才继续使用 Base64 适配。入库已有租约、重试、恢复和线性 pipeline，但不是具备分支/汇合/重放的通用 DAG 或跨区域队列；问答仍以 SSE 流式返回，不提供 Last-Event-ID 重放。

10. 项目表达：结论、因果、证据、取舍、边界

面试表达固定使用五步：

先给结论

- 讲因果链
- 给当前源码或测试证据
- 说明为什么不选其他方案
- 主动说尚未实现和下一步

不要堆砌技术名词，也不要吧路线图写成成果。能够展示的证据优先级是：可运行结果和指标 → 测试输出 → 源码调用链 → 设计文档。没有评测数字时直接说明没有，不编造“准确率提升 30%”。

10.1 当前可复述的一分钟版本

DeepSearcher 是一个可信 RAG 知识工作台。离线侧通过受控 LoaderRegistry 解析 PDF、Excel、PPTX 和图片 OCR，按结构化规则切分后用 `text-embedding-v4` 的 1024 维向量写入 Milvus；在线侧由 RAGRouter 在 DeepSearch、ChainOfRAG 和 NaiveRAG 中选择，再路由 Collection、迭代检索，并由当前配置的 `deepseek-v4-flash` 生成回答。产品具备 Workspace/KB 检索前授权、审计、可核对 Citation、反馈和 SSE；本地目录/已挂载 SMB 已完成首个同步纵切。它仍缺少第二个协作数据源、ACL 撤销同步、外部身份、分布式配额、压测和正式 SLO，不能包装成成熟生产平台。

10.2 验证口径：历史基线与当前状态

2026-08-02 的数字是历史基线，不代表当前工作树。P1-4. 2、P2 和 P3 的阶段记录分别保存了当时的完整门禁结果，最后一份 P3 为 ``1131 passed, 11 skipped``。文档刷新当天重跑 Python 全量测试得到 ``1130 passed, 11 skipped, 1 failed``；失败的 SSE 断连集成测试单独重跑通过，因此不可把这一次完整 Fast Gate 说成全绿。

最近一次阶段门禁和本次重跑的差异说明：测试数字必须附日期、命令与失败状态，不能只引用最大通过数。这些证据仍不等于模型质量 SLO、容量上限、完整无障碍合规或生产安全认证。

11. 第二阶段不能说错的事实

1. 本地代理上传的是文件内容，DeepSearcher `/load-files/` 接收的是后端路径。
2. Trace 是显式应用事件，不是模型隐藏思维链。
3. 产品问答通过 SSE 增量返回 v1 的公开阶段事件；完整执行 Trace 当前为 v7，旧 `/console` 的 `/api/query` 仍在完成后一次性返回 JSON Trace。
4. `latency_ms` 是代理等待后端的近似端到端时间，没有阶段 Span。
5. `consume_token` 主要是手工累计的 LLM 总 Token，不含完整 Embedding 计费。
6. 仓库有版本化业务评测、Trust/Citation/Risk/Provenance 金标与检索对照；结论必须限定在数据集、模型和运行配置。
7. 当前配置保持 Dense-only；Hybrid 已有对照评测，但没有胜出默认路径。
8. 默认 L2 是距离，越小越近，不能无条件称为相似度。
9. 同步路由在线程池中运行，不等于端到端异步或无限并发。
10. 全局共享客户端可以合理，但全局热更新和多 worker 配置不一致是风险。
11. 代理校验不是最终安全边界，直接后端必须重复校验与授权。

12. 独立 Worker、租约和线性 pipeline 提高了入库可恢复性，但不是通用 DAG 或持久化分布式任务队列。
13. 客户端断线不等于任务取消，SSE 也不保证 Exactly Once。
14. 当前产品状态和引用基础不能证明答案所有声明都 grounded。
15. 没有正式压测和质量指标，不能声称生产级高并发或准确率提升。
16. 当前默认页面是产品工作台；旧学习控制台只保留在 [/console](#)。
17. 产品上传使用 multipart；Base64 上传属于旧学习控制台接口。
18. 产品层已有 Workspace/KB ACL、成员组和审计；仍没有外部身份提供商、Organization 顶层或 Milvus 原生 ACL。